# Задача 1. Удаление дубликатов 2

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Дан массив $A$, в котором содержится $n$ целых чисел. Нужно удалить из него дубликаты (т.е. повторы чисел), так чтобы в массиве каждое имеющееся в нём значение встречалось ровно один раз.
Если значение встречается в массиве несколько раз, то нужно удалить все его вхождения, кроме самого первого. Порядок оставшихся элементов должен быть сохранён.
***Внимание:*** задачу требуется решать с помощью хеш-таблицы.

## Формат входных данных

В первых четырёх байтах записано число $n$ — сколько чисел в массиве ($1 \le N \le 10^6$).
Далее записано $n$ чисел, по четыре байта каждое. Все числа целые, по модулю не превышают $10^9$.

## Формат выходных данных

В первых четырёх байтах нужно вывести целое число $k$ — сколько различных чисел в массиве $A$. Далее нужно вывести $k$ этих чисел, по четыре байта каждое. Числа должны быть выведены в том порядке, в котором их первые вхождения идут в исходном массиве.

## Пример

| input.txt |
|---|
| 0A 00 00 00 01 00 00 00 01 00 00 00 FE FF FF FF<br>04 00 00 00 03 00 00 00 00 00 00 00 00 00 00 00<br>00 00 00 00 00 00 00 00 FE FF FF FF |
| **output.txt** |
| 05 00 00 00 01 00 00 00 FE FF FF FF 04 00 00 00<br>03 00 00 00 00 00 00 00 |

# Задача 2. Сумма чётных 2

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Требуется реализовать функцию foreach, которая пробегает по всем элементам заданного **массива** целых чисел и вызывает указанный callback для каждого из этих элементов.

Аналогичную функцию foreach нужно реализовать и для **связного списка**. Передаваемый callback должен поддерживать контекст.

Пример сигнатуры функции foreach для массива приведён ниже. Для связного списка реализуйте функцию foreach с аналогичной сигнатурой, чтобы можно было передавать в неё те же самые callback-функции.

```
// тип указателя на функцию, которую можно передавать как callback
typedef void (*callback)(void *ctx, int *value);
// здесь ctx -- это контекст, который передаётся в func первым аргументом
// последние два параметра задают массив, по элементам которого нужно пройтись
void arrayForeach(void *ctx, callback func, int *arr, int n);
```

Эти две функции требуется применить для решения следующей тестовой задачи: дана последовательность целых чисел, требуется посчитать сумму всех её чётных элементов.

Запишите данную последовательность как в массив, так и в связный список, и вызовите в каждом случае foreach. Функция-callback, которую вы передаёте в foreach, не должна обращаться к глобальным переменным: все необходимые для её работы данные передавайте через контекст.

## Формат входных данных

В первой строке содержится целое число $N$ — количество элементов последовательности ($1 \le N \le 100$). Во второй строке записано $N$ целых чисел через пробел — сама последовательность. Все элементы последовательности по абсолютной величине не превышают 100.

## Формат выходных данных

Выведите два целых числа: сумму всех чётных элементов последовательности. Первое число должно соответствовать сумме, вычисленной с помощью foreach по массиву, а второе — сумме, вычисленной с помощью foreach по связному списку.

# Пример

| input.txt | output.txt |
|---|---|
| 8<br>2 3 7 6 8 3 1 2 | 18 18 |

# Задача 3. Цикличность случайных чисел

| | |
|---|---|
| Источник: | базовая |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 2 секунды |
| Ограничение по памяти: | **разумное** |

Как известно, у генератора псевдослучайных чисел есть внутреннее состояние, которое может принимать конечное количество различных значений. Из этого следует, что если до статочно долго генерировать псевдослучайные числа, то в какой-то момент они начнут повторяться. В данной задаче нужно найти, с какого момента начнётся повторение у заданного квадратичного конгруентного генератора.

Квадратичный конгруентный генератор определяется четырьмя целочисленными параметрами $a$, $b$, $c$ и $M \geq 2$. Его состояние представляется целым числом state, которое всегда находится в диапазоне от 0 до $M - 1$ включительно. Функция перехода для этого генератора выглядит так:

```
uint64_t func(uint64_t s)
{
    return (s * s * a + s * b + c) % M;
}
```

Изначально, состояние генератора ***state*** равно единице. Далее каждый раз, когда пользователь запрашивает новое случайное число:

1. Пользователю выдаётся текущее значение state в качестве случайного числа.

2. К состоянию применяется функция перехода: state = func(state);

Обозначим последовательность случайных чисел, которую выдаёт генератор, через $x_0$, $x_1$, $x_2$, $x_3$,…. Нетрудно заметить, что всегда $x_0 = 1$. Будем говорить, что в этой последовательности циклически повторяется отрезок от $l$ до $r$, если $x_{l+i} = x_{r+i}$ для любого $i \geq 0$.

Даны параметры генератора, нужно найти отрезок от $l$ до $r$, который циклически повторяется. Поскольку вариантов выбора отрезка много, требуется найти такой, у которого число $r$ минимально возможное.

## Формат входных данных

В первой строке записано целое число $M$ — модуль генератора ($2 \leq M \leq 10^{12}$). Во второй строке записано три целых числа $a$, $b$, $c$ — параметры генератора ($0 \leq a, b, c \leq 10^9$).
Обратите внимание, что при указанных ограничениях в функции перехода func может происходить беззнаковое 64-битное переполнение. Это нормально, так и должно быть.

# Формат выходных данных

Выведите два целых числа $l$ и $r$ через пробел — отрезок, которые циклически повторяется. Среди всех возможных вариантов нужно выбрать тот, в котором число $r$ минимальное. Гарантируется, что в ответе $r \leq 2 \cdot 10^6$.

**Внимание:** для обнаружения совпадений нужно использовать хеш-таблицу.

# Примеры

| input.txt | output.txt |
|---|---|
| 11<br>1 4 5 | 1 4 |
| 999999999999<br>1 0 7 | 977966 1389969 |

# Задача 4. sql join

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Многие приложения используют базы данных для постоянного хранения информации.

На сегодняшний день наиболее популярны реляционные базы данных. В реляционной базе данные представляются в виде таблиц. Каждая таблица состоит из произвольного набора записей, каждая запись занимает одну строку таблицы. Запись можно воспринимать как структуру языка C: в ней есть фиксированный набор полей фиксированного типа. Каждый столбец таблицы содержит значение одного конкретного поля для всех записей. Таким образом, в таблице строки задают отдельные записи, а столбцы — поля этих записей.

Для извлечения и фильтрации данных из реляционной базы данных чаще всего используют язык SQL. При этом нередко пользователю базы данных нужно составить сборный отчёт по нескольким таблицам. В таком случае можно использовать операции соединения (join) таблиц. В данной задаче предлагается реализовать операцию Inner Join для двух конкретных таблиц.

Первая таблица содержит биографии известных актёров кино. Она была создана следующей командой SQL:

```
CREATE TABLE ActorBio (
Name varchar(30),
BirthYear int,
Country varchar(10)
);
```

Первый столбец называется Name и хранит имя актёра. Во втором записан год рождения как целое число. А третьем столбце записана страна, в которой жил актёр.

Вторая таблица содержит информацию о том, какой актёр в каких фильмах играл. Она была создана командой:

```
CREATE TABLE ActorInMovie (
ActorName varchar(30),
MovieName varchar(20)
);
```

Первый столбец содержит имя актёра ActorName, а второй — название кино, в котором этот актёр играл роль.

От вас требуется реализовать следующий SQL-запрос:

```
SELECT *
FROM ActorBio INNER JOIN ActorInMovie
ON ActorBio.Name = ActorInMovie.ActorName
```

Результатом этой операции является одна таблица, в которой пять полей: первые три поля взяты из таблицы ActorBio, а последние два — из ActorInMovie.

Механизм выполнения операции следующий:

1. Перебираем все пары записей $A$ и $B$, где $A$ взята из таблицы ActorBio, а $B$ — из таблицы ActorInMovie.

2. Для каждой пары проверяем условие соединения: что имя актёра Name в записи $A$ совпадает с именем актёра ActorName в записи $B$.

3. Если условие выполнено, то конкатенируем записи $A$ и $B$ и полученную запись с пятью полями добавляем в таблицу-результат.

Чтобы было проще понять, как работает операция, крайне рекомендуется посмотреть первый пример к задаче.

# Формат входного файла

В первой строке входного файла записано целое число $N$ — количество записей в таблице ActorBio ($1 \le N \le 10^5$). Далее идёт $N$ строк, в которых записаны записи таблицы ActorBio.

Затем записано целое число $M$ — количество записей в таблице ActorInMovie ($1 \le M \le 10^5$). Далее идёт $M$ строк, в которых записаны записи таблицы ActorInMovie.

Для каждой записи в файле записываются все её поля через пробел, в том порядке, в котором они определены. Все записи кроме года рождения строковые: они окружены символом двойной кавычки (ASCII 34) с обеих сторон. Каждое строковое значение непустое и может содержать в себе любые печатные символы ASCII (коды от 32 до 126 включительно) кроме символа двойной кавычки. Год рождения записан как целое число. Имена актёров не длиннее 30 символов, названия стран не длиннее 10 символов, названия фильмов не длиннее 20 символов.

# Формат выходного файла

Требуется вывести ровно $K$ строк: записи, получившиеся в таблице-результате после соединения. Каждая строка должна описывать одну запись с пятью полями, в том же формате, в котором эти записи записаны во входных данных. Порядок записей в выходном файле может быть любой.

Гарантируется, что $N \cdot M \le 10^5$.

# Примеры

| input.txt |
| --- |
| 8<br>"Peter Falk" 1927 "USA"<br>"Oleg Tabakov" 1935 "USSR"<br>"Andrei Mironov" 1941 "USSR"<br>"Arnold Schwarzenegger" 1947 "USA"<br>"Jean Reno" 1948 "France"<br>"Sharon Stone" 1958 "USA"<br>"Tom Cruise" 1962 "USA"<br>"Ryoko Hirosue" 1980 "Japan"<br>12<br>"Sharon Stone" "Basic Instinct"<br>"Jean Reno" "Mission: Impossible"<br>"Arnold Schwarzenegger" "Total Recall"<br>"Tom Cruise" "Mission: Impossible"<br>"Andrei Mironov" "Twelve Chairs"<br>"Sharon Stone" "Total Recall"<br>"Ryoko Hirosue" "Wasabi"<br>"Arnold Schwarzenegger" "Terminator"<br>"Jean Reno" "Wasabi"<br>"Peter Falk" "Colombo"<br>"Anatoli Papanov" "Twelve Chairs"<br>"Jean Reno" "Leon" |
| output.txt |
| "Andrei Mironov" 1941 "USSR" "Andrei Mironov" "Twelve Chairs"<br>"Arnold Schwarzenegger" 1947 "USA" "Arnold Schwarzenegger" "Total Recall"<br>"Arnold Schwarzenegger" 1947 "USA" "Arnold Schwarzenegger" "Terminator"<br>"Jean Reno" 1948 "France" "Jean Reno" "Leon"<br>"Jean Reno" 1948 "France" "Jean Reno" "Mission: Impossible"<br>"Jean Reno" 1948 "France" "Jean Reno" "Wasabi"<br>"Peter Falk" 1927 "USA" "Peter Falk" "Colombo"<br>"Ryoko Hirosue" 1980 "Japan" "Ryoko Hirosue" "Wasabi"<br>"Sharon Stone" 1958 "USA" "Sharon Stone" "Basic Instinct"<br>"Sharon Stone" 1958 "USA" "Sharon Stone" "Total Recall"<br>"Tom Cruise" 1962 "USA" "Tom Cruise" "Mission: Impossible" |

| input.txt |
| --- |
| 5<br>"O]V\| c -(SZ9mY ~‘/{8" 1950 "bo"<br>" Q G*u4 T:Eqy’d" 1979 "9"<br>"0" 2005 "jMsB"<br>" w cQ" 1982 "&"<br>" Q G*u4 T:Eqy’d" 2003 " J\|hL"<br>7<br>" Q G*u4 T:Eqy’d" "6q*EDh!"<br>"0" "s"<br>" Q G*u4 T:Eqy’d" "CQMG::dw{"<br>":D h%$ W^cr" "%‘De!Si"<br>" Q G*u4 T:Eqy’d" "]Zo"<br>" Q G*u4 T:Eqy’d" "t"<br>"0" "Uxb/.& " |
| output.txt |
| " Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "t"<br>" Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "CQMG::dw{"<br>" Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "]Zo"<br>" Q G*u4 T:Eqy’d" 1979 "9" " Q G*u4 T:Eqy’d" "6q*EDh!"<br>" Q G*u4 T:Eqy’d" 2003 " J\|hL" " Q G*u4 T:Eqy’d" "t"<br>" Q G*u4 T:Eqy’d" 2003 " J\|hL" " Q G*u4 T:Eqy’d" "CQMG::dw{"<br>" Q G*u4 T:Eqy’d" 2003 " J\|hL" " Q G*u4 T:Eqy’d" "]Zo"<br>" Q G*u4 T:Eqy’d" 2003 " J\|hL" " Q G*u4 T:Eqy’d" "6q*EDh!"<br>"0" 2005 "jMsB" "0" "Uxb/.& "<br>"0" 2005 "jMsB" "0" "s" |

# Задача 5. Лавинный эффект

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Для изучения качества хеш-функции принято рассматривать такое явление, как «лавинный эффект» (avalanche effect). Грубо говоря, он показывает, на какие биты хеша влияет каждый бит ключа. Считается, что если хеш-функция хорошая, то при переключении любого бита в ключе на противоположный в выходном хеше должна поменяться примерно половина битов. Например, хорошее лавинное поведение показывает хеш-функция Дженкинса. В этой задаче предлагается вычислить лавинный эффект для заданной хеш-функции.
Хеш-функция вычисляется следующим кодом на языке C:

```
uint64_t A, B, M, R, S;
uint32_t hashFunc(uint32_t x)
{
    return (((A * x + B) % M) % R) / S;
}
```

Как видно, эта функция принимает 32-битные беззнаковые ключи, и выдаёт 32-битные беззнаковые хеши. Осторожно: знаковость и битность всех переменных в этом коде имеет значение и должна быть именно такой, как написано!
Таблица лавинного эффекта для этой функции имеет размер 32 × 32, то есть в ней 32 строки и 32 столбца. В i-ой строке в j-ом столбце записано число, обозначаемое $p_{ij}$. Число $p_{ij}$ равно вероятности того, что изменение $i$-ого бита в случайном ключе приведёт к изменению $j$-ого бита в его хеше. При этом считается, что при выборе случайного ключа все 32-битные беззнаковые ключи равновероятны.

## Формат входного файла

В единственной строке записано пять неотрицательных целых чисел: $A$, $B$, $M$, $R$, $S$ — параметры хеш-функции. Обратите внимание, что все эти числа 64-битные и не превышают $10^{18}$. Чтобы избежать деления на ноль, параметры $M$, $R$, $S$ гарантированно ненулевые.

## Формат выходного файла

Выведите таблицу лавинного эффекта как 32 строки по 32 значения в каждой. Для каждой ячейки таблицы выведите число $p_{ij}$ в процентах: для этого нужно умножить $p_{ij}$ на 100 и округлить до целого.
Все выведенные числа должны быть целыми. Поскольку за короткое время не получится посчитать $p_{ij}$ с идеальной точностью, ваше решение может немного ошибиться: ваш ответ будет засчитан в том и только в том случае, если каждое число отличается от правильного не более чем на 1 (то есть ошибка в каждой вероятности должна быть не более процента).

# Примеры

| input.txt | output.txt |
|---|---|
| 2654435769 0 4294967296 4294967296 4096 | нормалёк =) |
| 2654435769 0 1048576 1000000000 1 | фу, неее =( |
| 938572893 2139875776 1000000007 1048576 1 | отлично! |
| 15642 322777666 10000000 1000000000 1 | как-то не очень... |

# Пояснение к примерам

В примерах входные данные не всегда умещаются в одну строку. Кроме того, выходные данные не указаны, так как они сильно большие и не входят в текст. Набор выходных данных для всех четырёх тестов можно скачать по ссылке.

По поводу хеш-функций из примеров можно сказать следующее:

1. В первом примере задана правильная хеш-функция Кнута, выдающая 20-битный хеш:

$$f(x) = \left[\frac{(A \cdot x) mod\ 2^{32}}{2^{12}}\right]$$

Как видно, лавинный эффект очень хороший: каждый бит хеша зависит хотя бы от 10-12 битов ключа. Единственная проблема — старшие биты ключа не влияют на младшие. Если кто-то решит хешировать числа, которые все делятся на 65536, то младшие 5 битов хеша не будут варьироваться вообще. С другой стороны, таких 32-битных чисел всего 65536, и, может быть, в хеш-таблице они разместятся без проблем.

2. Второй пример показывает неправильную реализацию хеш-функции Кнута, когда в хеш выбираются младшие биты вместо старших:

$$f(x) = (A \cdot x)\ mod\ 2^{20}$$

Как видно, у этой функции очень плохой лавинный эффект: на каждый бит хеша влияет намного меньше битов ключа, чем у правильной хеш-функции Кнута. В целом, эта хеш-функция показывает, почему нужно быть осторожным с модулем, являющимся степенью двойки.

3. Третий пример показывает правильный вариант линейной хеш-функции с 20 выходными битами:

$$f(x) = [(A \cdot x + B)\ mod\ P]\ mod\ 2^{20}$$

Здесь число $P$ равно 109 + 7 и является простым, а коэффициенты $A$ и $B$ выбраны произвольно. Заметьте, что перед взятием остатка от деления на $2^{20}$ предварительно берётся остаток от деления на простое $P$ — это важно.

У этой функции очень хорошее лавинное поведение: вся таблица заполнена. Хотя есть ячейки, в которых зависимость слабая (стоит почти ноль или почти 100), однако в общем каждый бит хеша зависит от подавляющего большинства битов ключа.

4. В последнем примере применяется такая же хеш-функция, но убрано взятие остатка от деления на простое число, отчего лавинный эффект резко ухудшается.

# Задача 6. sql join: хеширование

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | разумное |

Данная задача является продолжением задачи «sql join». Прочитайте условие оригинальной задачи перед тем, как читать дальше!

Допустим, в первой таблице $N$ записей, во второй — $M$ записей, а в таблице-результате $R$ записей. Легко видеть, что если имя актёра одинаковое во всех записях обеих таблиц, то в результате соединения в таблице окажется $R = NM$ записей. На практике такой случай довольно бессмысленный, обычно таблица-результат по количеству записей примерно такая же, как таблицы-аргументы: то есть $R = O(M+N)$. В таком случае для ускорения операции соединения следует использовать такие структуры данных и алгоритмы, чтобы выполнить соединение быстрее чем за $O(MN)$.

В данной задаче необходимо использовать хеш-таблицу для ускорения соединения. Запишите все записи одной таблицы в хеш-таблицу, в которой ключом является имя актёра. Затем переберите все записи другой таблицы: для каждой записи хеш-таблица позволяет быстро найти все совпадающие записи из первой таблицы. Входные/выходные данные в этой задаче такие же, как в задаче «sql join». Ограничения такие же ($1 \le N, M \le 10^5$), за одним важным исключением:

В этой задаче **не** гарантируется, что $N \cdot M \le 10^5$. Вместо этого гарантируется, что после соединения количество записей $R$ не превышает $10^5$.

# Задача 7. Найти коллизию

| | |
|---|---|
| Источник: | основная |
| Имя входного файла: | stdin |
| Имя выходного файла: | stdout |
| Ограничение по времени: | 3 секунды |
| Ограничение по памяти: | разумное |

В данной задаче вам предлагается найти коллизию для неизвестной вам хеш-функции, то есть указать два различных ключа, на которых значение хеш-функции совпадает.

Известно, что хеш-функция принимает 32-битное беззнаковое целое число на вход (ключ) и выдаёт 32-битное беззнаковое целое число на выход (хеш). Кроме того, известно, что хеш-функция очень хорошего качества.

Вы можете вычислять хеш-функцию на любых ключах, на каких хотите. Однако всего разрешается сделать не более $2 \cdot 10^5$ вычислений.

## Протокол взаимодействия

В данной задаче ваша программа будет работать не с файлами, а совместно с программой-интерактором. Ваша программа и интерактор будут запускаться одновременно, и соединяться пайпами (теми самыми пайпами, о которых упоминалось на лекции при рассмотрении очереди и кольцевого буфера). Всё, что ваша программа выводит в stdout, читает интерактор из своего stdin, а всё, что пишет интерактор на stdout, читает ваша программа из своего stdin.

Ваша программа должна печатать команды, которые интерактор будет выполнять. Есть два типа команд:

- Команда eval, после которой через пробел должно быть записано 32-битное беззнаковое целое число $X$. Эта команда предписывает интерактору вычислить значение хеш-функции от числа $X$. Интерактор вычислят его и записывает искомый хеш: ваша программа должна прочитать его из stdin как беззнаковое 32-битное целое.
- Команда answer, после которой через пробел должно быть записано два 32-битных беззнаковых целых числа $A$ и $B$. Этой командой ваша программа должна сообщить интерактору коллизию: числа $A$ и $B$ должны быть различными, но их хеш должен совпадать. После этого ваша программа должна сразу же завершить исполнение, ничего никуда больше не записывая и ничего ниоткуда не читая.

Если вы сделаете больше вычислений хеш-функции, чем разрешено, или выдадите неверный ответ командой answer, то ваше решение получит Wrong Answer.

Поскольку задача интерактивная, требуется:

1. Не открывать никаких файлов, не использовать freopen и fopen.

2. Писать команды с помощью printf и читать ответы с помощью scanf.

3. После каждой команды выводить символ перевода строки и сразу после этого выполнить: fflush(stdout);

Если вы забудете сделать команду fflush, то записанные вами в stdout данные останутсяв буфере, и никогда не попадут в пайп, а значит интерактор никогда их не получит и всё зависнет (вердикт Timeout).

Учтите, что в этой задаче все числа беззнаковые, так что писать и читать их надо сформатом "%u".

# Пример

| input.txt | output.txt |
|---|---|
| 2478003845<br>894250524<br>622810134<br>894250524 | eval 1<br>eval 2<br>eval 3<br>eval 4<br>answer 2 4 |

# Пояснение к примеру

Обратите внимание, что в примере сначала программа печатает команды в stdout, а уже потом на них приходят ответы от интерактора. В данном случае у ключей 2 и 4 получается одинаковый хеш, равный 894250524. Исполняемый файл интерактора вы можете скачать по ссылке (только для Windows). Чтобы запустить его просто поиграться, нужно использовать командную строку:

interactor.exe input.txt output.txt

# Задача 8. Сравнение подстрок

| | |
|---|---|
| Источник: | повышенной сложности |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 2 секунды |
| Ограничение по памяти: | разумное |

Дана строка $S$ длиной в $N$ символов, и размер блока $B$. В этой строке имеется ровно $(N-K+1)$ подстрок длины $B$ (подстрока — это часть строки, которая является непрерывным отрезком). Нужно раскрасить все эти подстроки в цвета, так чтобы одинаковые подстроки имели одинаковый цвет, а разные подстроки — разный цвет.

## Формат входного файла

В первой строке записано два целых числа: $N$ — длина строки и $B$ — длина рассматриваемых подстрок ($1 \le B \le N \le 10^6$).

Во второй строке дана сама строка $S$. Её длина равна $N$, и она состоит только из маленьких букв латинского алфавита.

## Формат выходного файла

Выведите в единственную строку $(N - B + 1)$ целых чисел через пробел: цвета всех подстрок длины $B$. Все цвета должны быть в диапазоне от 0 до $K-1$ включительно, где $K$ — количество различных цветов. Цвета нужно выводить в том порядке, в котором подстроки располагаются в строке $S$.

## Пример

| input.txt | output.txt |
|---|---|
| 15 3<br>abacabadabacaba | 0 3 1 5 0 4 2 6 0 3 1 5 0 |

# Задача 9. Цикличность случайных чисел+

| | |
|---|---|
| Источник: | повышенной сложности |
| Имя входного файла: | input.txt |
| Имя выходного файла: | output.txt |
| Ограничение по времени: | 1 секунда |
| Ограничение по памяти: | **5 мегабайт** |

Предлагается решить задачу про определение цикла квадратичного конгруентного генератора, используя $O(1)$ памяти. Очевидно, в таком решении не будет никакой хеш-таблицы.

Подробнее условие можно прочитать в задаче «Цикличность случайных чисел».

## Формат входного файла

В первой строке записано целое число $M$ — модуль генератора ($2 \le M \le 10^{12}$). Во второй строке записано три целых числа $a$, $b$, $c$ — параметры генератора ($0 \le a, b, c \le 10^9$).
*Обратите внимание*, что при указанных ограничениях в функции перехода func может происходить беззнаковое 64-битное переполнение. Это нормально, так и должно быть.

## Формат выходного файла

Выведите два целых числа $l$ и $r$ через пробел — отрезок, которые циклически повторяется. Среди всех возможных вариантов нужно выбрать тот, в котором число $r$ минимальное.

Гарантируется, что в ответе $r \le 2 \cdot 10^6$.

## Пример

| input.txt | output.txt |
|---|---|
| 11<br>1 4 5 | 1 4 |
| 999999999999<br>1 0 7 | 977966 1389969 |